Compilers Assignment

Guided by, Dr Jagdish Chakole
Performed by, Ayush Dhoble - BT23CSE204

Objective : Make a toy-compiler for a hypothetical language that follows all phases of Compiler from lexical analysis to Target Code Generation.

Phases of Compilers :

- Lexical Analysis
- Syntax Analysis
- Semantic Analysis
- Intermediate Code Generation
- Code Optimization
- Target Code Generation

Proposed Approach

To achieve the objective, we designed and implemented a small custom programming language and built a complete compiler for it using **Flex, Bison** and **C**. The language was kept intentionally simple so that all major compiler phases could be demonstrated clearly without making the project unnecessarily complex.

The compiler accepts source code written in the toy language, processes it through the standard compilation pipeline, and produces multiple forms of output. These include the parsed **AST**, the **optimized AST**, **three-address code**, **target code**, and the **final program output** after execution. This makes the project suitable for demonstrating both compilation and interpretation concepts.

Language Design

Instead of using the exact syntax of C, a separate toy language `Mina` was created with its own keywords and structure. The language supports:

- `proc` defines a function
- `let` declares a variable
- `set x <- expr` assigns a value

- `when expr { ... } otherwise { ... }` is conditional control flow
- `repeat expr { ... }` is looping
- `give expr;` returns from a function
- `show(expr);` prints a value
- `ask()` reads a numeric value from standard input

This helped in showing that the compiler is not just reusing an existing language, but actually handling a newly defined grammar.

Major Components of the Compiler

1. Lexical Analysis

The source program is first scanned by the lexer. It converts the raw text into meaningful tokens such as keywords, identifiers, literals, operators, and punctuation symbols. This stage removes whitespace and comments and prepares the input for parsing.

2. Syntax Analysis

The parser checks whether the token sequence follows the grammar of the language. If the syntax is valid, it builds an **Abstract Syntax Tree (AST)**, which represents the structure of the program in a tree form.

```
Ayush@DESKTOP-ONOSB3R MINGW64 /d/compiler/toy_compiler_project
$ ./toycompiler sample.mina
==== AST ====
PROGRAM
  FUNC square : num
    PARAM x : num
    BLOCK
      GIVE
        BINOP *
          VAR x
          VAR x
  FUNC main : void
    BLOCK
      SHOW
        STRING Enter a number:
      LET n : num
        CALL ask
      SHOW
        STRING Square:
        CALL square
          VAR n
      WHEN
        BINOP ==
          BINOP %
            VAR n
            NUM 2
          NUM 0
        BLOCK
          SHOW
            STRING Even number
      OTHERWISE
        BLOCK
          SHOW
            STRING Odd number
```

3. Semantic Analysis

After parsing, the compiler checks whether the program is logically correct. This includes verifying variable declarations, function usage, type consistency, and scope rules. If any semantic error is found, the compiler reports it with the line number.

4. Intermediate Code Generation

The AST is then converted into **three-address code**, which is a simpler intermediate representation. This form breaks complex expressions into smaller steps and makes later stages easier to handle.

```
==== 3 ADDRESS CODE ====
FUNC square
  t1 = x * x
  return t1
ENDFUNC square
FUNC main
  print #0
  t2 = read()
  n = t2
  print #0
  t3 = n % #2
  t4 = t3 == #0
  ifz t4 goto L1
  print #0
  goto L2
L1:
  print #0
L2:
ENDFUNC main
```

5. Code Optimization

A basic optimization is applied to improve the intermediate representation. Simple optimizations such as constant folding are used to reduce unnecessary computation while preserving the meaning of the program.

```
==== OPTIMIZED AST ====
PROGRAM
  FUNC square : num
    PARAM x : num
    BLOCK
      GIVE
        BINOP *
          VAR x
          VAR x
  FUNC main : void
    BLOCK
      SHOW
        STRING Enter a number:
      LET n : num
        CALL ask
      SHOW
        STRING Square:
        CALL square
          VAR n
      WHEN
        BINOP ==
          BINOP %
            VAR n
            NUM 2
          NUM 0
        BLOCK
          SHOW
            STRING Even number
      OTHERWISE
        BLOCK
          SHOW
            STRING Odd number
```

6. Target Code Generation

The optimized code is then translated into a low-level pseudo-assembly form. This target code is not real machine code, but it closely resembles instruction-level execution and helps demonstrate the final stage of compilation.

```
==== TARGET CODE ====
FUNC square
    LOAD x
    LOAD x
    MUL
    STORE t1
    LOAD t1
    RET
END square
FUNC main
    PUSH 0
    PRINT
    READ t2
    LOAD t2
    STORE n
    PUSH 0
    PRINT
    LOAD n
    PUSH 2
    MOD
    STORE t3
    LOAD t3
    PUSH 0
    EQ
    STORE t4
    LOAD t4
    JZ L1
    PUSH 0
    PRINT
    JMP L2
L1:
    PUSH 0
    PRINT
L2:
END main
```

7. Runtime Execution

Finally, the compiler executes the program and displays the actual output. This allows the entire pipeline to be demonstrated from source code to execution result.

```
Ayush@DESKTOP-ONOSB3R MINGW64 /d/compiler/to
$ cat sample.mina
proc square(x: num) : num {
    give x * x;
}

proc main() : void {
    show("Enter a number:");
    let n: num <- ask();

    show("Square:", square(n));

    when (n % 2 == 0) {
        show("Even number");
    } otherwise {
        show("Odd number");
    }
}
```

```
==== PROGRAM OUTPUT ====
Enter a number:
> 6
Square: 36
Even number

Ayush@DESKTOP-ONOSB3R MINGW64 /d/co
```

Symbol Table

```
==== SYMBOL TABLE ====
Name            Type        Kind
main            void        FUNC
square          num         FUNC
ask             num         FUNC
==== OPTIMIZED AST ====
```

Outcome

The completed project demonstrates a full compiler workflow for a custom programming language (Mina). It shows how source code is converted step by step into increasingly lower-level representations until it reaches executable output. The project successfully covers all major compiler phases and gives a practical understanding of compiler construction.